

Předmět: Programování v C/C++ (B6B36PCC)

Semestrální práce na téma: násobení matic.

Student: Vladimír Zubkov

Checklist pro odevzdání

- Základní implementace:

1. Kód

- Obsahuje váš kód CMakeLists.txt přes který se vaše semestrálka dá postavit? **Ano**
- Obsahuje váš kód implementaci optimalizovanou pro jedno vlákno? **Ano**
- Je implementována nápověda (argument programu -help)? **Ano, a command-line**

2. Práce s vlákny pro vyšší bodové hodnocení

- Používá váš kód více vláken když má k dispozici více jader? **Ano, podle počtu logických jader**
- Nepoužívá váš kód rozšíření jazyka? (Například OpenMP, VLA) **Ne, nepoužívá**
- Nepoužívá váš kód nepřenosné knihovny (Například POSIX, Win32, filesystem, windows.h, conio). **Ne, nepoužívá**

3. Výsledky měření

- Proběhlo měření nad optimalizovaným programem? (Byla binárka zkompileována v Release módu?) **Ano, v debug a release modu**
- Vyšel běh využívající více vláken rychlejší? **Nad úplně klasickým násobením ano, nad optimalizovaným ne**

4. Zpráva

- Obsahuje vaše zpráva hash commitu (nebo tag), vůči kterému byla napsaná? **V dopise, tato prezentace už bude na gitlabu**
- Obsahuje vaše zpráva popis zadání? **Na další stránce**
- Obsahuje vaše zpráva popis naměřených výsledků? **Ano, na konci zprávy**
- Obsahuje vaše zpráva popis prostředí, ve kterém jste prováděli měření? **Na této stránce níže**

- Použité algoritmy: a) klasické násobení, b) optimalizované klasické násobení, c) vícevláknové optimalizované násobení, d) blokové násobení podle předpokládané velikosti cache L1, e) Strassenův algoritmus.

- Environment:

```
PC: Processor Intel(R) Core(TM) i5-7300U CPU @ 2.60GHz 2.71 GHz
Logical Processors 4, L1 cache 128Kb
Installed RAM 8,00 GB (7,84 GB usable)
System type 64-bit operating system, x64-based processor
Plugged into an outlet
```

```
Compiler: GNU GCC
Version: 13.1.0, zkompileováno jak v RELEASE tak i DEBUG modu.
Kontrola memory leaků provedená v Microsoft Visual Studio 2022
pomoci CRT ztráty paměti neobjevila.
```

- Struktura prezentace: na začátku obecně popíšeme problematiku násobení matic a vyjmenujeme algoritmy, potom popíšeme použité v semestrálce algoritmy, a předvedeme analýzu jejich výstupů.

Popis úlohy

Násobení matic má velmi široké uplatnění. Bez nich se neobejdete v počítačové grafice při transformacích a projekcích, ve strojovém učení pro výpočty neuronových sítí, v simulacích fyzikálních procesů nebo při řešení rovnic a konvolucí funkcí. Najdeme ho také v kryptografii, při zpracování obrazu, v ekonomických modelech, robotice a analýze dat. Je to základní nástroj pro práci s komplexními systémy a velkými daty.

Klasický výpočet velkých matic, při kterém se prvky násobí řádek po řádku a následně se sčítají má velkou asymptotickou složitost $O(n^3)$, to vyžaduje velký výkon procesoru a mívá velké nároky na paměť. Používání čísel s plovoucí desetinnou čárkou přináší problémy s přesností kvůli hromadění zaokrouhlovacích chyb. Jsou aplikace, u nichž se například potřebné koeficienty objevují u hlavní diagonály, a ostatní prvky jsou nulové (například ve fyzice a mechanice, modelování Markovových řetězců, řešení diferenciálních rovnic numerickými metodami, při zpracování signálů a obrazů, atd.), násobení každého prvku takových matic je potom neefektivní.

Dále, při paralelním zpracování a v distribuovaných systémech vznikají potíže se synchronizací a přenosem dat. Navíc, nesoulad rozměrů matic znemožňuje jejich přímé násobení bez předchozího zpracování.

Bylo navrženo několik řešení těchto problémů. Pro snížení výpočetní složitosti byly vyvinuty algoritmy, jako je *Strassenův algoritmus* a algoritmus *Coppersmitha-Winograda*, které zlepšily asymptotickou složitost na $O(n^{\log_2 7})$ a $O(n^{2.376})$. Pro řídké matice se používají optimalizované formáty ukládání, jako *CSR* nebo *CSC*, které ukládají pouze nenulové prvky. Paralelní a distribuované algoritmy, například *Cannonův algoritmus*, umožňují efektivní zpracování velkých matic na výkonných clusterech nebo GPU. Pro zlepšení číselné stability se využívají algoritmy s dvojitou přesností a minimalizací zaokrouhlovacích chyb. U neslučitelných rozměrů matic se před násobením provádí jejich předzpracování, například doplněním nulových řádků nebo sloupců. Přibližné metody, jako jsou algoritmy Monte Carlo, pak nabízejí rychlejší řešení na úkor malé chyby ve výsledku.

V naší práci použijeme klasické násobení matic jako benchmark a následně různé druhy jeho optimalizace. S nimi potom porovnáme Strassenův algoritmus.

1. Klasické násobení matic

Nechť A je matice o rozměrech $m \times k$ a B je matice o rozměrech $k \times n$. Výsledná matice C má rozměry $m \times n$ a její prvek c_{ij} se počítá jako:

$$c_{ij} = \sum_{r=1}^k a_{ir} \cdot b_{rj}.$$

Asymptotická složitost klasického násobení matic je $O(m \cdot n \cdot k)$.

2. Optimalizované klasické násobení matic

Optimalizace se týká jen zlepšení přístupu k paměti a optimalizace smyček.

Podle vzorce výše je vidět, že v klasickém algoritmu se prvky matice A a B načítají z paměti opakovaně. Optimalizace zahrnuje například a) “zploštění” cyklu při vynulování prvků c_{ij} , které se vynulují rovnou celým řádkem, a tak se nemusí vynulovat pokaždé; b) změnu algoritmu přechodu po prvcích matic A , B a C tak, aby se načítaly po řádcích, a využili tak cache procesoru, při opětovném přístupu je velká věrohodnost že se dotaz “nemine” cache, a data se načtou rychle.

3. Vícevláknové optimalizované klasické násobení matic

Tento algoritmus se liší od předchozího tím, že navíc využívá paralelní výpočty s více vlákny (multithreading). Každé vlákno zpracovává určitou část řádků matice A , přístup k řádkům matice B je sekvenční a navazuje na matici A . Počet vláken je dán počtem logických jader procesoru. Hlavní vlákno počká na dovršení práce každého vlákna, sjednotí a předá výsledek.

4. Blokované násobení matic

Blokované násobení matic si rovněž jako i optimalizovaná klasická metoda, zakládá na využití cache procesoru. Matice se ale rozdělí do čtvercových bloků určité velikosti. Doporučována velikost, odvozena například od velikosti L1 cache je:

$$\text{blockSize} = \sqrt{\frac{\text{L1 cache size}}{\text{sizeof(double)}}}$$

Při realizaci semestrální práce bylo podmínkou dodržení nezávislosti kódu na platformě, a tak jsem opustili od nápadu zjistit velikost cache počítače dynamicky. Použili jsme blok o velikosti strany 36, což by mohlo stačit pro L1 cache velikosti cca 10Kb ($36^2 \cdot 8/1024$). My jsme měli velikost 128Kb na čtyřech logických jádrech, tj. 32Kb na každé jádro.

Pokud strana matice není násobkem velikosti bloku, na konci se použije blok, jehož prvky nepřekračují hranice matice, nebo jeho velikost je oříznuta.

Tento algoritmus byl realizován na jednom vlákně.

5. Strassenův algoritmus

Strassenův algoritmus pro násobení matic zlepšuje efektivitu tím, že matici rozděluje na menší bloky a používá sedm rekurzivních multiplikací místo tradičních devíti. Tento přístup snižuje časovou složitost z $O(n^3)$ na $O(n^{\log_2 7})$.

V případě nečtvercových matic se použije doplňování nulami, aby matice měly čtvercový tvar, přičemž se velikost matice rozšiřuje na nejbližší mocninu dvojky. Tím se zajistí, že algoritmus může fungovat i na nečtvercových maticích.

Realizace

Program má formu příkazové řádky. Na vstupu má definovanou minimální velikost strany matice a krok, k tomu jako omezující podmínku – maximální sečtenou velikost řádků nebo sloupců obou násobených matic. Při počátku výkonu algoritmu se velikost matic zvyšuje po krocích tak, aby byla zachována plynulost navýšení jejich celkové velikosti, tj.:

```
1. krok: A[řádky X sloupce] * B[řádky X sloupce]
2. krok: A[řádky + krok X sloupce] * B[řádky X sloupce]
3. krok: A[řádky - krok X sloupce] * B[řádky X sloupce + krok]
4. krok: A[řádky + krok X sloupce] * B[řádky X sloupce + krok]
5. krok: A[řádky - krok X sloupce + krok] * B[řádky + krok X sloupce - krok]
```

V uvedeném algoritmu se řádky a sloupce počítají za konstanty, tj. na 1. a na 5. kroku jsou si rovné, jejich rozdíly jsou “odkrokováné” kroky. Každý takový krok je násoben různým výše popsáním způsobem: klasicky, optimalizovaně, vícevláknově, blokově a pomocí Strassenovy metody.

Poté se algoritmus opakuje na nové úrovni, dokud nedosáhne stropu v podobě ustanovené maximální sečtené velikosti řádku a sloupců obou násobených matic.

Uvedený postup se stává zjevný ihned při spuštění algoritmu. Výchozí nastavení programu je minimální strana 500, krok 500, celková velikost 3000. Toto stačí pro jeden výše uvedený cyklus – pět kroků po 5 algoritmech, celkem se do výsledného CSV souboru uloží 25 řádků a příslušné statistiky.

Velikosti počáteční strany, kroku a omezení se dají měnit v příkazové řádce, rovněž je možné spouštět jednotlivé algoritmy podle příslušného přepínače na příkazové řádce. Jednotlivé příkazy – nastavení matic a vybrané algoritmy se dají v příkazové řádce kombinovat. Výsledek běhu algoritmu a statistiky se uloží do souboru **results.csv** ve stejném adresáři. Po každém spuštění programu se tento soubor přepíše.

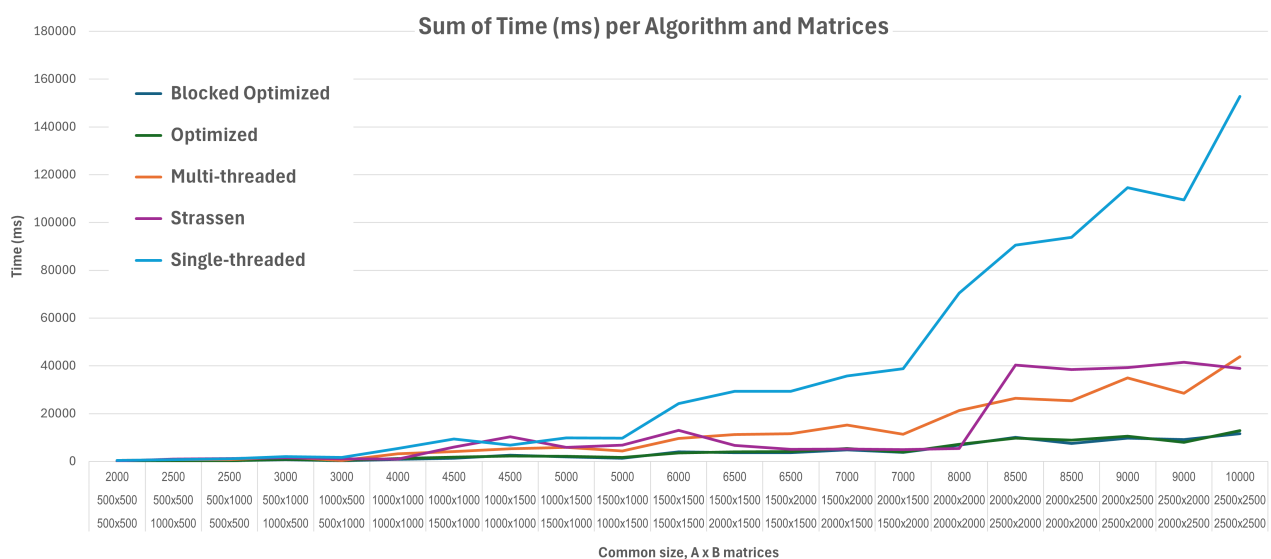
Kromě toho je možné spouštět testy porovnání výsledných matic z různých, i vybraných algoritmů. Výsledky porovnání se vypíší do konzole. Způsoby ovládání programu se vypíší v příkazové řádce pomocí příkazu **-help**.

Program v práci využívá dočasné soubory násobených matic a násobky matic v testech, ukládané na disk ve stejném adresáři. Po dokončení běhu programu se tyto soubory vymažou.

Program byl zkompileován jak v režimu **Debug** tak i **Release**. Rozdíl ve prospěch **Release** ve výkonu a objemu paměti na disku je markantní, circa desetkrát.

Výstupy a analýza

Spustil jsem algoritmus na maticích s počáteční stranou velikosti 500, krokem 500 a celkovým omezením 10000 řádků či sloupců. Výsledky jsou znázorněné na následujícím obrázku.



Obrázek 1: Porovnání rychlosti jednotlivých algoritmů

Je vidět, že algoritmy se vedou téměř podobně až do 5 tisíc jednotek (řádků či sloupců), následně klasické násobení rychle zaostává. Obdivuhodně obě jednovláknové optimalizované metody, které využívají cache procesoru, se vedou velice podobně a velice dobře – předbíhají vícevláknovou realizaci konstantní rychlostí (na rozdíl od klasické metody, která zřejmě zaostává za nimi exponenciálně). O tom, která metoda z optimalizovaných je lepší, není rozhodnuto. Vícevláknová realizace má zřejmě větší časové nároky kvůli potřebě správy vláken, kromě toho, na 90 % využívá procesor, oproti 35 % využití procesoru jednovláknovými realizacemi. Nakonec, Strassenova metoda se vede rovněž velice dobře, na stejné úrovni s optimalizovanými metodami, ale při počtu 8500 jednotek zřejmě přechází na novou větší velikost submatic, a tak její rychlost skokově narůstá, v bodě předbíhá i vícevláknovou realizaci, přitom ale s počtem velikosti zpracovaných jednotek zůstává poměrně konstantní a vícevláknová realizace ji v počtu stráveného času nahání. Statistiky, na kterých se zakládá uvedený graf, jsou uvedené na konci prezentace v tabulce (1).

Závěr

Z uvedené analýzy je tedy zřejmé, že při násobení matic CPU je potřeba využívat algoritmů, které využívají cache procesoru, a následně, na těchto algoritmech si zakládat další metody.

Z uvedení dále tabulky je rovněž zřejmé, že matice podobných dimenzí je vždy, pokud je to možné, násobit většími stranami, tj. je lepší, když levá matice má větší počet sloupců, než řádků při všem ostatním stejném, a opačně platí i pro matici B – větší počet řádků než sloupců je obvykle lepší. Toto souvisí se způsobem algoritmického zpracování dat.

Použité zdroje

1. Умножение матриц: эффективная реализация шаг за шагом.
<https://habr.com/ru/articles/359272/>
2. Strassen, V. (1969). Gaussian Elimination is not Optimal. *Numerische Mathematik*, 13(4), 354–356.

Tabulka 1: Naměřené hodnoty délek násobení matic různými metodami (ms, kromě velikostí)

MatrixSizeA	MatrixSizeB	TotalSize	Optimized	Block Opt	Strassen	Multi-threaded	Single-threaded
500x500	500x500	2000	102	128	174	448	405
1000x500	500x500	2500	253	268	944	798	762
500x500	500x1000	2500	312	235	1169	802	1042
1000x500	500x1000	3000	869	745	1567	1829	2078
500x1000	1000x500	3000	427	294	1106	454	1684
1000x1000	1000x1000	4000	1195	860	996	3217	5443
1500x1000	1000x1000	4500	1820	1292	5958	4137	9431
1000x1000	1000x1500	4500	2303	2613	10309	5256	6869
1500x1000	1000x1500	5000	2148	1860	5869	5901	9883
1000x1500	1500x1000	5000	1715	1309	6827	4344	9734
1500x1500	1500x1500	6000	3539	3960	12983	9574	24155
2000x1500	1500x1500	6500	3961	3654	6645	11273	29324
1500x1500	1500x2000	6500	4148	3651	5073	11635	29372
2000x1500	1500x2000	7000	5431	4874	5200	15239	35748
1500x2000	2000x1500	7000	3870	3724	4916	11352	38783
2000x2000	2000x2000	8000	7164	6808	5386	21283	70409
2500x2000	2000x2000	8500	9772	10072	40278	26422	90470
2000x2000	2000x2500	8500	8899	7529	38439	25324	93824
2500x2000	2000x2500	9000	10533	9756	39294	34979	114627
2000x2500	2500x2000	9000	7981	9169	41474	28540	109424
2500x2500	2500x2500	10000	12946	11666	38888	43796	152722